

Syllabus : Programmation en C (1ère année de DTS)

Ce syllabus est conçu pour initier les étudiants de première année de DTS (Diplôme de Technicien Supérieur) aux concepts fondamentaux de la programmation à travers le langage C. L'objectif est de leur donner une base solide en logique algorithmique et en manipulation des données, essentielle pour leur future carrière en informatique.

Objectifs généraux du module

- Acquérir les bases du raisonnement algorithmique.
 - Comprendre la syntaxe et les structures du langage C.
 - Maîtriser les concepts de base de la programmation (variables, boucles, fonctions, etc.).
 - Savoir structurer et écrire un programme en C lisible et efficace.
 - Se familiariser avec l'environnement de développement et le processus de compilation.
-

Module 1 : Introduction à la programmation (5 heures)

- **1.1. Qu'est-ce que la programmation ?**
 - Définition d'un algorithme et d'un programme.
 - Le rôle du compilateur et de l'éditeur de liens.
 - **1.2. Premiers pas en C**
 - Structure de base d'un programme C (main, #include).
 - Afficher des messages à l'écran (printf).
 - Le processus de compilation d'un programme simple.
-

Module 2 : Les bases du langage C (10 heures)

- **2.1. Variables et types de données**
 - Déclaration et initialisation des variables.
 - Les types de données fondamentaux (entiers, flottants, caractères).
 - Les constantes.
- **2.2. Les opérateurs et expressions**

- Opérateurs arithmétiques, relationnels et logiques.
 - La priorité des opérateurs.
 - **2.3. Les entrées/sorties simples**
 - Lire des données au clavier (scanf).
 - Formatage des données.
-

Module 3 : Les structures de contrôle (10 heures)

- **3.1. Les structures conditionnelles**
 - L'instruction if, if-else et switch.
 - Les opérateurs de comparaison.
 - **3.2. Les boucles**
 - L'instruction while et do-while.
 - La boucle for.
 - **3.3. Contrôle de flux avancé**
 - Les instructions break et continue.
 - Les boucles imbriquées.
-

Module 4 : Les fonctions et le découpage de code (15 heures)

- **4.1. Le concept de fonction**
 - Rôle des fonctions dans la modularité du code.
 - Déclaration, définition et appel de fonctions.
 - Les paramètres et la valeur de retour.
- **4.2. La portée des variables**
 - Variables locales et globales.
 - Les passages de paramètres par valeur.
- **4.3. Découper un programme**
 - Utilisation des en-têtes (.h) et des fichiers sources (.c).
 - Le processus de compilation de plusieurs fichiers.

Module 5 : Les tableaux et les chaînes de caractères (10 heures)

- **5.1. Les tableaux**
 - Déclaration et initialisation des tableaux.
 - Accéder aux éléments d'un tableau.
 - Les tableaux multi-dimensionnels.
- **5.2. Les chaînes de caractères**
 - Les chaînes de caractères comme tableaux de caractères.
 - Fonctions de manipulation de chaînes (<string.h>).

Module 6 : Les pointeurs (15 heures)

- **6.1. Introduction aux pointeurs**
 - Qu'est-ce qu'un pointeur et à quoi il sert ?
 - Déclaration, opérateurs * et &.
- **6.2. Les pointeurs et les fonctions**
 - Le passage de paramètres par adresse.
 - Les pointeurs comme valeur de retour.
- **6.3. Allocation dynamique de mémoire**
 - Fonctions malloc, calloc et free.
- **6.4. Pointeurs et tableaux**
 - La relation entre pointeurs et tableaux.
 - Les pointeurs sur tableaux et les tableaux de pointeurs.

Module 7 : Les structures et les fichiers (10 heures)

- **7.1. Les structures**
 - Définir une structure (struct).
 - Accéder aux membres d'une structure.
- **7.2. Les fichiers**

- Le concept de flux de données.
 - Ouvrir, lire, écrire et fermer un fichier.
 - Entrées/sorties formatées.
-

Méthode d'enseignement et évaluation

- **Méthode d'enseignement** : Le cours alterne entre la théorie et la pratique. Chaque concept sera illustré par des exemples de code et des exercices de programmation à réaliser en TP (Travaux Pratiques). L'accent sera mis sur la rédaction de code propre et commenté.
- **Évaluation** :
 - **Contrôle continu (50%)** : Évaluations régulières sur table et en salle de TP.
 - **Projet final (50%)** : Un projet de programmation plus complexe (par exemple, un jeu de type "pendu" en console ou un petit programme de gestion de contacts) qui intègre les concepts vus dans le syllabus.